

Resource-Efficient Continual Learning for Sensor-Based Human Activity Recognition

CLAYTON FREDERICK SOUZA LEITE and YU XIAO, Aalto University, Finland

Recent advances in deep learning have granted unrivaled performance to sensor-based **human activity recognition (HAR)**. However, in a real-world scenario, the HAR solution is subject to diverse changes over time such as the need to learn new activity classes or variations in the data distribution of the already-included activities. To solve these issues, previous studies have tried to apply directly the continual learning methods borrowed from the computer vision domain, where it is vastly explored. Unfortunately, these methods either lead to surprisingly poor results or demand copious amounts of computational resources, which is infeasible for the low-cost resource-constrained devices utilized in HAR. In this paper, we provide a resource-efficient and high-performance continual learning solution for HAR. It consists of an expandable neural network trained with a replay-based method that utilizes a highly-compressed replay memory whose samples are selected to maximize data variability. Experiments with four open datasets, which were conducted on two distinct microcontrollers, show that our method is capable of achieving substantial accuracy improvements over baselines in continual learning such as Gradient Episodic Memory, while utilizing only one-third of the memory and being up to 3× faster.

CCS Concepts: • **Human-centered computing** → **Ubiquitous and mobile computing**; • **Computing methodologies** → **Neural networks**;

Additional Key Words and Phrases: Continual learning, human activity recognition, deep learning

ACM Reference format:

Clayton Frederick Souza Leite and Yu Xiao. 2022. Resource-Efficient Continual Learning for Sensor-Based Human Activity Recognition. *ACM Trans. Embedd. Comput. Syst.* 21, 6, Article 85 (October 2022), 25 pages. <https://doi.org/10.1145/3530910>

1 INTRODUCTION

Sensor-based **human activity recognition (HAR)** refers to the problem of inferring the type of actions an individual is performing from the readings of wearable sensors such as accelerometers and gyroscopes. It finds several applications in healthcare [38], smart home systems [10], virtual or augmented reality games [19], sports training [15], fall detection [17], etc. Even though the advent of **deep learning (DL)** has enabled unprecedented recognition performance in HAR without the need for feature engineering [7, 29, 41], HAR faces important challenges that are yet under-explored. In particular, after being deployed, the HAR solution is subject to the two following dynamic conditions.

Authors' address: C. F. S. Leite and Y. Xiao, Aalto University P.O. Box 15400, FI-00076 Aalto, Espoo, Finland; emails: {clayton.souzaleite, yu.xiao}@aalto.fi.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

1539-9087/2022/10-ART85 \$15.00

<https://doi.org/10.1145/3530910>

- (1) Additional activity classes are required to be recognized. This is named a **class-incremental** scenario. An example could be a smart home system that permits the users to customize their interaction via new gestures (can be regarded as activities).
- (2) Variations in how the same activity is performed. We term this dynamic condition a **style-incremental** scenario. The intra-activity variability appears due to differences in the characteristics of users. For instance, in the case of a HAR system designed for employee training, as the employees become more skillful at their job, they are expected to perform the activities at a faster pace with lower physical effort.

These varying conditions are often unpredictable and may not be possible to be addressed before the deployment of the HAR system. Therefore, this demands a flexible HAR solution capable of continuously accommodating new knowledge (i.e., new classes or new styles) while preserving previously acquired ones. Simply training the HAR classifier with new data leads to catastrophic forgetting - i.e., a situation where old knowledge is abruptly lost [22]. It is possible to preserve old knowledge by periodically completely re-training the classifier with all the data that were previously utilized. However, this process may not be able to be performed on wearable devices for two distinct reasons. Firstly, it requires storing in memory an ever-increasing amount of data. Secondly, re-training the classifier all over again is a time-consuming process. As an alternative, the re-training process can be executed in the cloud, whereas this raises privacy and security issues related to user data in cases where end users generate new data to better adapt the HAR solution to their specific needs [9]. Furthermore, re-training is not a gradual process where a HAR classifier progressively adapts to new knowledge. Rather, it is a periodic process that requires a good amount of new data to be collected before re-training the classifier in question. This non-continuous characteristic forces the user to interrupt the use of the HAR solution for a considerable amount of time while it is being updated.

The field of machine learning that studies methods to substitute complete re-training without catastrophic forgetting is termed **continual learning**. Previous studies on continual learning focused mostly on computer vision applications with the aim of maximizing accuracy, regardless of resource efficiency [3, 5, 8, 28, 35, 37]. Since HAR involves user data, to prevent data privacy and security issues associated with operations in the cloud, it is essential that methods can be executed locally, such as on low-cost embedded devices. This requires the solutions to be computationally efficient.

In this work, our key contribution is a **computationally-light, memory-efficient** and simple-to-implement continual learning method for HAR to address both class-incremental and style-incremental scenarios. The novelties of our method are two-fold and deeply founded on the particularities of HAR. As our first contribution, we find that the use of highly compressed - via downsampling and precision reduction - rehearsal data greatly contributes towards the goal of memory efficiency and low computational expense while circumventing catastrophic forgetting. Second, we propose to carefully select the rehearsal data to maximize its diversity with respect to ways of performing the activities. To the best of our knowledge, our work is the first to address continual learning for HAR in resource-constrained devices. Note that differently from conventional studies on porting DL to resource-constrained devices [23, 25, 34, 44] for inference only, our method focuses on on-device training in a continual learning setting. We have performed experiments on four distinct public datasets for both aforementioned scenarios and are considering six different continual learning baselines for comparison. The results have evidenced that our proposal - compared to the second best-performing baseline [28] - is capable of delivering up to 19.6% higher accuracy while utilizing approximately $3\times$ less memory and providing up to a $3.5\times$ faster training duration.

Table 1. Summary of Comparisons Between Diverse Works on Deep Learning-based HAR, Including Ours

Work	Focus	Novelty
InnoHAR [43]	Improving prediction performance	A neural network architecture based on inception neural networks
ST-DeepHAR [1]	Improving prediction performance	An architecture that combined attention mechanisms, convolutions, recurrent units and residual connections
Zhang et al. [51]	Improving prediction performance	Transformed sensor data into images and processed the images with traditional computer vision recognition networks
Zheng [52]	Improving prediction performance	An architecture that extracted local and global features distinctively
Jin et al. [21]	Improving prediction performance and recognizing unknown classes	An Euclidean-based loss function instead of the conventional cross-entropy
Wu et al. [42]	Improving prediction performance and applying the HAR system to the interaction with social robots	An end-to-end system for interacting with robots via hand gestures
Zhang et al. [44]	Optimizing resource efficiency during inference	Constrained the weights of the networks to only three distinct values
Leite and Xiao [25]	Optimizing resource efficiency during inference	Proposed to avoid overlapping windows and favoring coarse-grained features over fine-grained ones at certain moments
AHAR [34]	Optimizing resource efficiency during inference	A dynamic neural network architecture that reduced in size when needed
Leite and Xiao [26]	Optimizing resource efficiency during inference	A method for detecting and removing redundant sensor channels from the input
MDLdroidLite [50]	Optimizing resource efficiency during training and inference	An architecture that grew in size when required, with its growth smartly controlled
Ours	Optimizing resource efficiency during training and inference for continual learning scenarios	An architecture that grows in size when new classes need to be learned, a compressed replay memory with carefully selected samples as to increase variability

The rest of this work is organized as follows. In Section 2, the related work is introduced. In Section 3, we delineate our method. Section 4 details the setup of the experiments conducted in Section 5, where the results are presented. Section 6 discusses the advantages and limitations of our proposal. Finally, Section 7 concludes the work.

2 RELATED WORK

In this section, we first briefly mention recent studies on deep learning for HAR and its porting to embedded devices - summarized in Table 1. Then, we describe previous studies on continual learning - most of which were originally applied to computer vision tasks - as well as their suitability to be implemented for HAR on resource-constrained devices (Table 2). All the continual learning methods discussed below can be ported to HAR and employed in both class- and style-incremental scenarios, with the sole exception that Progressive Neural Networks [37] are not adequate for the style-incremental scenario without modifications. We classify the literature into three distinct types: regularization-based methods, replay methods, and parameter isolation methods [8]. Regularization-based methods employ an additional loss term whose minimization aims at

Table 2. Comparison Between Our Work and the Previous Studies Related to Continual Learning

	Ours	EWC [40]	LwF [27]	iCaRL [35]	GEM [28]	AGEM [5]	PNN [37]
Classification performance	High	Medium	Low	Medium	High	Medium	Low
Memory footprint	Low	High	Medium	Medium	High	Medium	Low
Computational expense	Low	High	Medium	Medium	High	Medium	Very low

preserving previously acquired knowledge. Replay methods utilize a small subset of stored samples to retrain the neural network. On the contrary, parameter isolation methods segment the neural network into a portion responsible for storing old knowledge and a portion assigned for learning new knowledge.

2.1 Deep Learning-based HAR

The advent of deep learning has driven a paradigm shift in HAR not only by enabling a more painless development of HAR solutions without the need for hand-engineered features but also by delivering more accurate results [13]. For the past few years, considerable research endeavor has been directed towards developing ever more accurate deep learning-based algorithms, mostly by exploring more intricate and sophisticated neural network architectures. InnoHAR [43] ported the concept of inception neural networks to HAR. Abdel-Basset et al. [1] introduced ST-DeepHAR that combined attention mechanisms, convolutional neural network, recurrent neural networks, and residual connections for feature extraction. Zhang et al. [51] transformed sensory data into pseudo-images and employed the U-Net for the classification of activities. Zheng [52] proposed to split the sliding window containing the activity into several sub-windows, from which local spatio-temporal features were learned with an attention-based convolutional neural network module. Global spatio-temporal features were learned in parallel with cascaded convolutions. Both global and local features were then fused by concatenation and passed through a dense layer to generate predictions. In [21], the conventional cross-entropy loss for classification tasks was dropped in favor of a loss function based on the Euclidean distance. Such loss function aimed at minimizing the Euclidean distance between high dimensional embedding vectors originated from inputs pertaining to the same class, while maximizing their distance in the opposite case of inputs with distinct classes. This setting allowed the classification of not only the activities seen in the training phase, but also unknown ones - i.e. classes seen for the first time at test time. In a more application-focused work, Wu et al. [42] utilized LSTMs in the prediction of hand gestures from estimated 3D hand poses, with very high recognition performance. These hand gestures were then employed in the interaction with robots.

The drawback of deep learning-based solutions is their resource-intensive nature rendering their deployment onto resource-constrained devices often infeasible. Running the training of the deep models and their inference on the cloud is an alternative. However, it brings privacy-related issues and possible additional overheads due to latency in communication [25]. For this reason, the development of resource-efficient HAR powered by deep learning has attracted attention from the research field. Zhang et al. [44] introduced a training scheme that constrained the weights of the neural network to only three possible values, resulting in memory savings of 11× and computational costs down by 9×. Performance close to that of a full-precision model was achieved. In [25], resource efficiency was attained by rethinking deep learning for HAR. The authors proposed to avoid overlapping sliding windows and to skip fine-grained features in favor of rough ones on certain occasions. In their method, the performance was shown to be comparable to heavier models. In AHAR [34], the authors approached the resource efficiency problem with an adaptive neural network. During the inference phase, controlled by a decision algorithm, either a portion

of the neural network or its entirety can be utilized to predict the activity from the input data. Computational resources were spared in the former case. Experiments showed that comparable performance was achieved with respect to the baseline. In [26], redundant and irrelevant sensor channels in the input data were analyzed and removed, thus resulting in a more lightweight input and, consequently, a more resource-efficient solution. While our method concentrates on resource efficiency in a continual learning training scenario, it is also lightweight for the inference phase and can potentially be combined with the previously mentioned approaches for greater resource efficiency at this phase.

MDLdroidLite [50] appeared as the first resource-efficient solution oriented towards both training and inference. Similar to our method, it followed the concept of a neural network that incrementally grows in size during the training. However, in MDLdroidLite, the growth was controlled by a **Model Predictive Control (MPC)** optimization algorithm that minimized the neural network's structure and its loss function simultaneously. Modifying a neural network's architecture during its training leads to a large variance of neurons after each growth, which results in slow convergence in circumstances where the training data is available indefinitely. In a continual learning scenario - i.e. our case - such a large variance introduces an even larger catastrophic forgetting effect since it directly alters the distribution in each of the neural network's layers. Although the authors of MDLdroidLite proposed a mechanism to alleviate the large variance issue, it did not completely eliminate it. In our design, the neural network grows by instantiating blocks in parallel that only merge at the last layer (to be discussed in more detail in Section 3), thus avoiding the large variance of neurons. In this work, we concentrate on a resource-efficient solution for both on-device training and inference in the special scenario of continual learning. The following subsections present the recent efforts in continual learning with a glance at their resource efficiency.

2.2 Regularization-based Methods

Elastic Weight Consolidation (EWC) [40] approaches continual learning with the Bayesian framework by requiring the posterior of previous tasks to be the prior of the new task. As the old posterior is intractable, it is approximated by a Gaussian distribution with a mean equal to the network parameters learned at the end of the previous task and with diagonal precision equal to the diagonal of the Fisher information matrix. This results in an additional loss term that penalizes changes in the weights important to the old tasks.

Li and Hoiem introduced **Learning without Forgetting (LwF)** [27], which uses knowledge distillation [18] for continual learning. Prior to training a new task, the outputs of the neural network given the new data are obtained and saved. These outputs are utilized during the training when knowledge is distilled with the additional loss term proposed in [18].

Inspired by the continual learning ability of biological neural networks, Zenke et al. [49] proposed **Synaptic Intelligence (SI)**. In SI, a regularization penalty similar to EWC [40] is employed. However, differently than EWC, the importance of each weight for the previously learned tasks is computed online over the entire training trajectory. Despite the difference, both approaches provide similar performance.

In **Memory Aware Synapses [2] (MAS)**, a regularization loss term similar to EWC [40] and [49] was introduced. When learning a certain task, MAS computes the importance of each weight in the neural network based on how sensitive the predicted output is to a change in the value of the weight. During the training of a new task, MAS penalizes changes to the importance weights for the previous task. In some cases, MAS outperformed other regularization-based methods as EWC [40], LwF [27], and SI [49].

Jha et al. [20] presented an empirical analysis on regularization-based methods for HAR. The authors observed that, in general, these methods fall short in terms of performance. Since

regularization protects weights important to previous tasks and HAR neural networks are often made lightweight to fit into resource-constrained devices, new and old tasks potentially compete for the same weights resulting in a trade-off between performances on the new and old tasks [39]. Such a trade-off can be addressed by increasing the complexity of the neural network as a future-proofing method. However, starting off with a fairly complex classifier translates into unnecessarily high resource utilization. Moreover, this can lead to overfitting in the early tasks.

In this work, we introduce an extensible neural network that is able to grow progressively as new knowledge is required to be learned. With this, we aim at avoiding the trade-off of regularization-based methods without resorting to using a complex neural network as future-proofing.

2.3 Replay Methods

In the **Incremental Classifier and Representation Learning (iCaRL)** method, Rebuffi et al. [35] explored continual learning in an incremental class learning scenario. The authors proposed to use episodic memories of equal size for each class previously learned. In these memories, the selected samples are those with the smallest Euclidean distances to the mean of the learned features of the class in question. The designed loss function was made of two terms. The first term consists of the common cross-entropy on the new classes, whereas the second term employs knowledge distillation [18] on the previously learned classes. Originally proposed for class-incremental scenarios, iCaRL can be also adapted for task incremental settings, as in [8].

In **Gradient Episodic Memory (GEM)** [28], the problem is formalized as a constrained optimization where the goal is to minimize the loss for the new task while avoiding any increase - but allowing the decrease - of the loss for the previous tasks, which is calculated on an episodic memory. To reduce the memory and computational resource utilization, instead of utilizing the entire episodic memory, **Averaged GEM (AGEM)** [5] uses only a subset of it randomly sampled at every training step. In both methods, the samples of a certain task in the episodic memory are simply chosen to be the last ones used to train the neural network for that task.

Gradient-based Sampling Strategy (GSS) [3] selects samples according to their gradients with the goal of maximizing the sample diversity in the gradient space of the episodic memory. The method was shown to deliver as high as or superior accuracy compared to other sample selection methods such as GEM [28] and iCaRL [35].

Deep Generative Replay (DGR) by Shin et al. [39] explored Generative Adversarial Networks [12] in continual learning. The method revolves around generating a multitude of fake replay data that mimic previous training examples, thus reducing the large memory requirements involved in storing real replay data. During the training of new tasks, the generated data are used as a rehearsal to avoid catastrophic forgetting. The method also combines regularization methods as LwF [27] for better results.

Ye et al. [46] benchmarked diverse proposed architectures of GANs for generating artificial rehearsal data in HAR. In general, these architectures outperformed common continual learning methods as iCaRL [35], except in cases when the dimension of the generated data was high. In a typical GAN setting for continual learning, during the training of a certain task k , considerable computational costs are dispensed to perform forward passes in the $k - 1$ generators that produce artificial data for the rehearsal of previous tasks. Forward and backward passes in the generator of the current task k , as well as its discriminator, are also required. The presence of these $k + 1$ additional neural networks would also induce significant memory utilization, which may defeat the purpose of avoiding the storage of large amounts of real replay data. Furthermore, despite recent advances in GANs [32], the instability of their training is still an open issue that requires several rounds of trial and error hyper-parameter tuning. Hence, the employment of GANs in an online continual learning HAR system is impractical for low-cost resource-constrained devices.

Ye and Callus [45] worked on an extensible neural network to learn an increasing number of activities in HAR assisted with GEM [28]. To accommodate new knowledge associated with the new activities, their neural network undergoes a layer addition process with carefully initialized weights to avoid catastrophic forgetting as a consequence of the expansion. When training on new activities, old knowledge is preserved with GEM [28]. Their approach is not only computationally expensive due to the use of GEM [28], but also as a result of the procedure that preserves the knowledge of the neural network during expansion - i.e. catastrophic forgetting has to be avoided also during the expansion. Our extensible neural network does not require two stages of forgetting avoidance, but a single one. Furthermore, to prevent forgetting, we utilize a simple and computationally economical method.

2.4 Parameter Isolation Methods

Rusu et al. [37] introduced expandable neural networks that grew in size whenever a new task needed to be learned - **Progressive Neural Networks (PNN)**. In PNNs, tasks do not share the same neural network weights. Instead, before learning of a new task, the PNN is expanded laterally and the weights associated with previous tasks are fixed. PNNs employ lateral expansions to the neural network in such a manner that prior knowledge can be leveraged during the learning of a new task. Despite the fact that the computational graph of PNNs grows exponentially in complexity due to the lateral connections, the resource utilization can still stay low since the gradients are only computed with respect to a portion of the neural network. The biggest issue with PNNs is their performance on class-incremental scenarios due to the absence of an effective method to counteract forgetting. Moreover, due to the lateral connections, the implementation of forward and backward passes in PNNs is not an easy programming exercise in computing devices that do not support deep learning frameworks like TensorFlow.

In this work, we abstain from lateral connections by proposing an extensible neural network whose expansions are not linked to previous blocks. To better combat forgetting in an extensible neural network, we propose to use a downsampled and precision-reduced low-cost replay memory, while avoiding freezing portions of the neural network.

PackNet [30] explores iterative pruning to counteract catastrophic forgetting. The method consists of two distinct phases: training and pruning. Initially, the entire neural network is trained for the first task. At the end of the training, PackNet searches for unimportant network weights for the task. These weights are pruned and the remaining ones are kept fixed. The pruned weights are then trained for the second task, after which another round of pruning is performed before the training for the following task. This process continues indefinitely.

Similar to the regularization-based methods, as more tasks are learned, a trade-off where old and new tasks compete for weights appears in PackNet. As mentioned previously, our work proposes an extensible neural network to circumvent this issue.

3 METHODS

3.1 Definitions

We consider two cases of online continual learning: **class-incremental** and **style-incremental**. In the class-incremental case, a neural network f_θ - with parameters (weights and biases) θ - has to learn new classes continuously from a stream of data formed by samples represented as a 2-tuple (x_i, y_i) , where x_i is the i -th sample (in HAR, a sample consists of a sliding window) of the stream and y_i is the corresponding class label. In the style-incremental case, the neural network f_θ has to learn to classify a fixed number of classes from a stream of data whose style of performing the activities varies continuously.

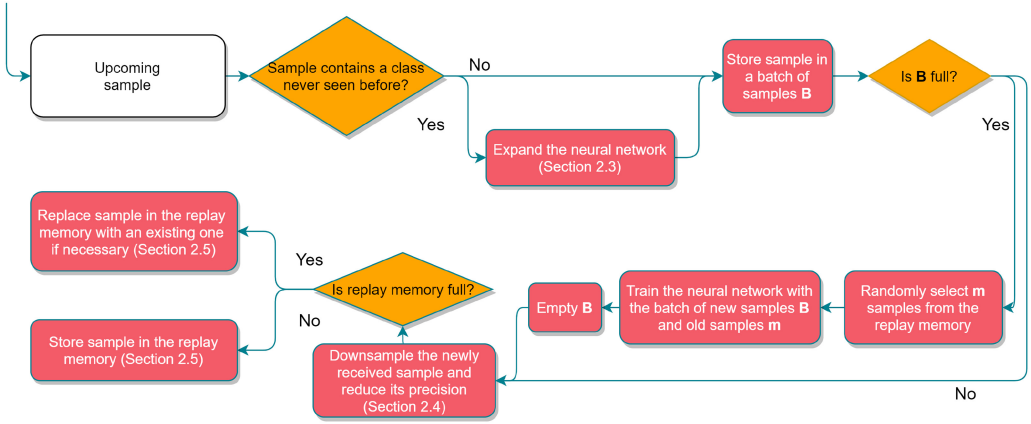


Fig. 1. Overview of our method.

In the class-incremental case, a task τ is defined as the task of learning to classify the new classes introduced in the stream of data of τ together with the previously learned classes. For the style-incremental case, the task τ refers to learning to classify a static number of classes with the style introduced in the stream of data of τ in addition to the previously seen styles. We also consider that we have access to a limited non-zero portion of the previous samples through a replay memory $\mathcal{M}$ of a fixed size M .

3.2 Overview of the Method

Figure 1 illustrates the entire process of our continual learning method for both class- and style-incremental cases. For every upcoming sample (x_i, y_i) , it is first checked whether the class y_i has been seen before. If not, the neural network is expanded to accommodate the new class. The sample is then stored in a batch of samples $\mathcal{B}$. After this step, if $\mathcal{B}$ has not yet reached a pre-defined limit size, the sample is downsampled and has its precision reduced (see Figure 3). In the opposite case, if $\mathcal{B}$ is full, before modifying the sample by downsampling and precision reduction, m randomly selected samples from the replay memory $\mathcal{M}$ in addition to those from $\mathcal{B}$ are utilized to train the neural network before $\mathcal{B}$ is emptied. Once the sample is modified (by downsampling and precision-reduction), it is directly stored into $\mathcal{M}$, if $\mathcal{M}$ has not yet reached its limit in size. In case the replay memory is full, the decision problem of whether or not replacing an existing sample (x_k, y_k) in the replay memory with the newly arrived sample (x_i, y_i) - where $k < i$ - must be solved (to be discussed in Section 3.5). In the following subsections, we delineate the details of the neural network, the modification of the samples as well as the motivation behind it, and the management of the replay memory.

3.3 Neural Network

Our neural network (Figure 2) is composed of three distinct stages: (1) the spatial feature extraction stage, (2) the temporal feature extraction stage, and (3) the classification stage. To extract spatial features - i.e. features that arise from dependencies among the sensors - we utilize inception blocks of n convolutional learnable filters (where n is a hyper-parameter) as proposed by Xu et al. [43]. However, instead of four, we employ two inception blocks sequentially since we have not witnessed any appreciable performance improvement with a higher number of blocks. The output of the second inception block is flattened to a 1-dimensional vector before entering a **long short-term memory (LSTM)** layer - with n hidden units - for temporal feature extraction. The

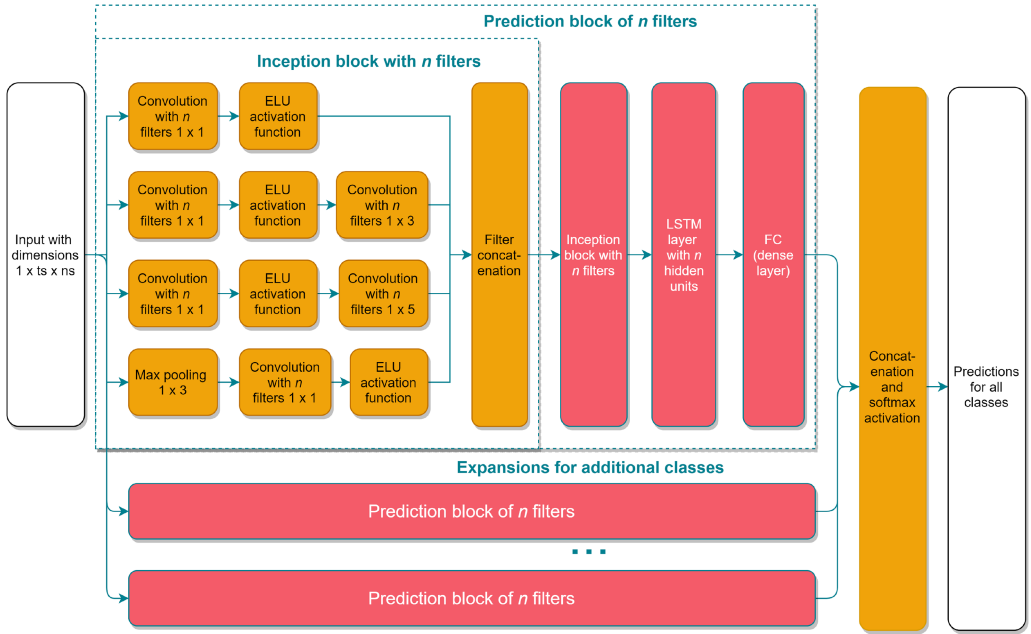


Fig. 2. Structure of our expandable neural network.

temporal features pass through a **fully connected (FC)** - or dense - layer and a softmax activation function to produce class-wise probabilities. This entire trajectory forms a prediction block $\mathcal{P}$ of n filters.

In the beginning, the neural network consists of one sole prediction block $\mathcal{P}_0$ and delivers probabilities for an initial number of classes c_0 , which also means that the FC layer of $\mathcal{P}_0$ is composed of c_0 neurons. When a new class is required to be learned, a new neuron is added to the FC layer if $c_0 + 1 \leq C_{\mathcal{P}_0}$, where $C_{\mathcal{P}_0}$ is a hyper-parameter that determines the maximum number of classes for the prediction block $\mathcal{P}_0$. If $c_0 + 1 > C_{\mathcal{P}_0}$, then a new prediction block $\mathcal{P}_1$ is instantiated. This procedure occurs as long as new classes need to be learned.

The new instances of the prediction block are always added in parallel. To group the outputs of the FC layers from individual blocks, a simple concatenation is utilized before the softmax activation function. We use the cross entropy as the loss function to train the neural network.

3.4 Modifying the Sample

Given that the computing device has a limited memory space for the replay memory, we propose to downsample and reduce the precision of the samples (Figure 3) before storing them into the replay memory. Our premise is that a greater number of downsampled and lower-precision samples is more beneficial for avoiding catastrophic forgetting in HAR than a lower number of unmodified samples - this will be verified in Section 5.3. While it is true that the compression of the samples leads to the loss of information, Section 5.3 demonstrates that this is not a drawback for the continual learning purposes. Decimation is used as the downsampling method reducing the sampling rate to as low as 4Hz. Even though 4Hz is far below the recommended data sampling rate for a HAR system [24], we argue that it is sufficient for the purpose of circumventing catastrophic forgetting. Note that only the samples in the replay memory are downsampled and have their precision reduced.

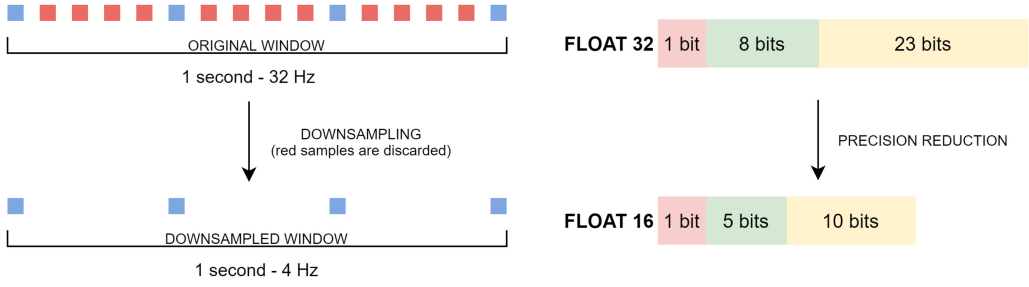


Fig. 3. The downsampling (left) and precision-reduction (right) procedures. A float32 value is formed by 32 bits, where 1 bit is reserved for indicating negative or positive values (signed bit - in red color), 8 bits are used for the exponent (in green color), and 23 bits are assigned to the significand (in yellow).

The precision is reduced by casting all the values in the sample from single-precision (i.e. *float32*) to half-precision (*float16*). In practice, this operation rounds a few decimal places of all values in the sample. Before utilizing the samples from the replay memory to train the neural network, they are upsampled with cubic interpolation, and zeros are added as trailing decimal places to cast the values back to single-precision. The cubic interpolation method was selected among other interpolation variants empirically.

3.5 Replay Memory

Since the same activity can be performed in various distinct ways, it is important that the neural network retains the knowledge related to this intra-activity variability when accommodating new knowledge. Since in our method the replay memory is at the core of alleviating forgetting, it is essential that the set of samples that populate the replay memory presents a rich variability in activity execution. The question that naturally arises is how to ensure intra-activity variability in the replay memory at a low computational cost. We propose to address this problem by establishing the premise that this variability rises as time passes - that is, we assume that two consecutive samples have lower variability than distant ones. Therefore, ideally, the samples in the replay memory should be as far apart from each other as possible.

At the beginning of the first task τ_1 , when the replay memory $\mathcal{M}$ is not yet full, every sample that arrives is directly stored in it - after the downsampling and precision-reduction modification - as a 3-tuple (x, y, u) , where u is a unique identifier - represented as an integer number - such that consecutive samples have consecutive identifiers. When $\mathcal{M}$ reaches its limit M in size, an upcoming sample s' is only stored if there exists a sample s in $\mathcal{M}$ such that its replacement with s' maximizes the spread between identifiers in the replay memory. Mathematically, we denote the replay memory as a set $\mathcal{M} = \{(x_k, y_k, u_k) \mid k \in \{1, 2, \dots, M\}\}$ and the spread of the identifiers in $\mathcal{M}$ as $d_{\mathcal{M}}$ (Equation (1)).

$$d_{\mathcal{M}} = \sum_k \min_{h \neq k} |u_k - u_h| \quad (1)$$

If there exists more than one sample s whose replacement with s' maximizes $d_{\mathcal{M}}$, the oldest one is chosen to be replaced. When the second task τ_2 starts, half of the samples in the replay memory are deleted. The samples to be deleted are those whose absence in the replay memory maximizes $d_{\mathcal{M}}$. After the deletion, the replay memory is segmented into two equal parts - $\mathcal{M}_1$ and $\mathcal{M}_2$ - of size $M/2$, where the $\mathcal{M}_1$ only contains the samples of the previous task and $\mathcal{M}_2$ is empty in order to store samples of τ_2 . This is done to promote task balance in the replay memory. The samples of

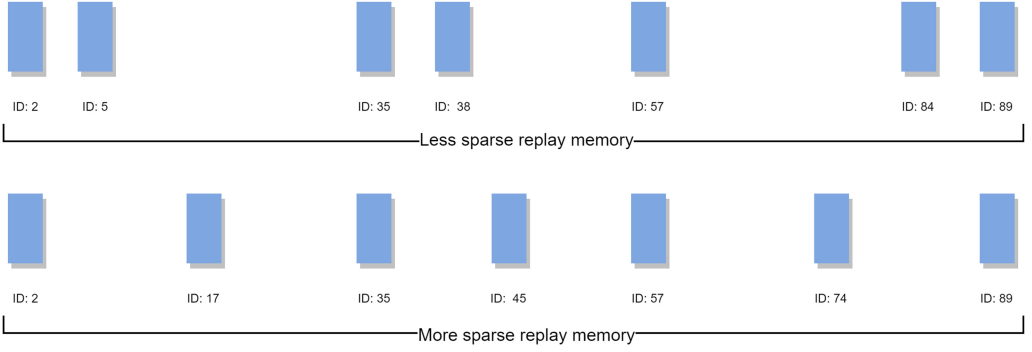


Fig. 4. Different replay memories represented by placing their samples on an x-axis where the position of each sample is given by its identifier. On the top, the samples are less sparsely distributed regarding their identifiers. The purpose of maximizing d_M (Equation (1)) is that of making the samples more sparsely distributed.

τ_2 are then stored in M_2 until it is full. After M_2 has reached its limit $M/2$ in size, a new sample s' is replaced with $s \in M_2$ if it maximizes d_{M_2} . This process continues indefinitely.

Intuitively, the maximization of Equation (1) allows for a replay memory with sparsely distributed identifiers since it signifies maximizing the minimum absolute difference between neighboring samples (Figure 4). Since we impose that consecutive (recorded one after the other in time) samples possess consecutive identifiers, having a replay memory whose sample identifiers are sparsely distributed leads to higher intra-activity variability.

An imbalanced task representation in the replay memory can lead to selective forgetting, i.e. the under-represented tasks in the replay memory can be forgotten more easily. Our assumption that intra-activity variability rises with time is plausible since the factors that lead to this variability are driven by time. A few examples of these factors are (1) an increasing level of skill in performing a certain activity; (2) injuries or physical impairments; (3) sensor placement drift and aging; and (4) tiredness can affect the way the user performs a certain activity. Observe that our sample selection algorithm involves mainly addition and subtraction operations on integers, thus being extremely light-weight compared to sample selection methods such as GSS [3] that includes additional backward passes on the neural network.

4 EXPERIMENTAL SETUP

4.1 Datasets

4.1.1 PAMAP2. PAMAP2 [36] is a daily-life activity dataset collected from nine different participants wearing a heart rate monitor and IMUs on the chest, right hand, and left ankle. The dataset is comprised of 18 daily-life activities. However, following previous works [14, 24–26], to avoid heavy imbalance during the training, six of the activities - which are rarely present in the data - are discarded. The 12 remaining activities considered in this work are composed of: lying quietly, sitting, standing, ironing, vacuum cleaning, ascending stairs, descending stairs, walking, Nordic walking, bicycling, running, and rope jumping. The null class - made of transient activities - is also discarded. Originally recorded at 100Hz, the data is downsampled by decimation to 33.3Hz since a faster sampling rate does not improve the performance for this dataset [24], but incurs additional computation and memory costs.

4.1.2 Opportunity. The Opportunity dataset [6] is formed by 18 kitchen-specific activities: null class, cleaning a table, opening/closing the fridge, opening/closing the dishwasher, opening/closing three different drawers (at different heights), opening/closing two different doors,

toggling lights on and off, and drinking from a cup. The data were collected at 33Hz from four participants equipped with accelerometer-only sensors and IMUs. Since the null class is present in more than 70% of the data, we opted to discard it. Otherwise, the inclusion of the null class would result in an extreme class imbalance where each of the remaining classes would represent on average less than 1.8% of the dataset.

4.1.3 Skoda. This dataset [47] comprises 10 activities related to a car assembly line scenario: writing down notes, opening/closing the engine hood, checking door gaps, opening/closing door, opening/closing two doors, checking the trunk gap, opening/closing the trunk, and checking the steering wheel. The data are formed by a sole participant that wore five accelerometers on each arm. Each accelerometer was sampled at an original rate of 100Hz, which was later decimated to 33Hz, for the same reason as in PAMAP2.

4.1.4 MHEALTH. MHEALTH [4] contains 12 daily-life activities of varying heart rate zone: standing still, sitting and relaxing, lying down, walking, climbing stairs, bending waist forward, elevating arms frontally, bending knees, cycling, jogging, running, and jumping to the front & back. Collected from 10 participants, the data consist of 50Hz IMU readings placed on the chest, right wrist, and left ankle of the subject; as well as measurements from a 2-lead electrocardiogram also at 50Hz.

Note that the considered datasets - even after the exclusion of some classes in the case of PAMAP2 and Opportunity - still present a realistic degree of imbalance.

4.2 Data Pre-Processing

Prior to the training, the data were normalized to zero mean and unit variance. We segmented the data utilizing the conventional sliding window approach, where each window is labeled with the activity present for the majority of its duration. Following [16, 24–26, 33, 48], we segmented the PAMAP2 dataset with sliding windows of 5.12 seconds with 78% overlap. For the remaining datasets, the duration of the windows was set to one second with 50% of overlap. The split of the datasets into training, validation, and test set is given according to the scenario.

4.2.1 Class-Incremental Scenario. For PAMAP2, following the same protocol as in [14, 16, 25, 26, 43], we utilized the data from subject 5 for the validation set, and those of subject 6 for the test set. The data of the remaining subjects were assigned to form the training set. As for Opportunity, the validation set consisted of run 2 from subject 1, where the test set was formed by runs 4 and 5 from subjects 2 and 3 [14, 16, 25, 43]. Again, the remaining data were allocated to the training set. For Skoda, since the dataset is composed of one sole subject, the training, validation, and test set were randomly selected with proportions 0.70, 0.15, and 0.15, respectively. Finally, for MHEALTH, the data from subjects 8 and 9 constituted the validation and test sets, respectively.

4.2.2 Style-Incremental Scenario. To account for different styles, we consider that each participant in the dataset has a different style. Skoda and Opportunity were not utilized in this scenario due to their low number of subjects. Hence, in this scenario, PAMAP2 and MHEALTH were split into training, validation, and test sets with the proportions of 0.6, 0.2, and 0.2, respectively.

4.3 Evaluation Metrics

We denote as $R_{i,j}$ the test classification accuracy on task τ_j after being trained for task τ_i . In our class-incremental setting, the test classification accuracy on a task τ accounts for all classes learned during task τ as well as before. Similarly, in the style-incremental setting, the test classification accuracy on a task τ accounts for all styles learned before and in the course of task τ .

We utilize the **average accuracy (ACC)** (Equation (2)) and **backward transfer (BWT)** (Equation (3)) performance metrics as proposed in GEM [28]. The ACC metric accounts for the performance across all tasks. The BWT metric measures the influence that learning a task τ_j has on a previously learned task τ_i , that is, how knowledge is propagated to earlier tasks. Positive backward transfer indicates that learning a certain task increases the performance on a previously learned task, whereas (large) negative backward transfer suggests (catastrophic) forgetting. Additionally, we also report the sequence of values $R_{i,j} \forall i = j \leq T$ - with T as the total number of tasks.

$$\text{ACC} = \frac{1}{T} \sum_{i=1}^T R_{T,i} \quad (2)$$

$$\text{BWT} = \frac{1}{T-1} \sum_{i=1}^{T-1} R_{T,i} - R_{i,i} \quad (3)$$

In addition to performance evaluation, we also provide resource consumption evaluation. For this, we utilize three additional metrics: memory usage, the elapsed time of the training, and inference time. The memory usage measures the maximum amount of occupied memory in the system during the training of the continual learning method; whereas the elapsed time of the training consists of the interval of time required to execute the training of the neural network. Note that, in the calculation of the memory usage, we have discarded the memory utilized by other processes running in the system that are not related to the continual learning method. The inference time accounts for the duration of a forward pass on the neural network.

4.4 Code Implementation

Our method - as well as the baselines for comparison - was implemented in Python 3.8.8 with TensorFlow 2.5.0. We ran both training and inference on two embedded devices: an ARM Cortex-A72 SoC (Raspberry Pi 4B microcontroller) and an ARM Cortex-A53 SoC (Raspberry Pi 3B+ microcontroller). Adam was utilized as the optimizer in all methods with a learning rate equal to $5e-4$, $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The LSTM layer had a dropout rate set to 0.2. For all datasets, the samples were downsampled by $8\times$ before entering the replay memory. The size B was set to 16 as well as the number of samples in the replay memory m to be trained at once, hence resulting in an overall batch size of 32. To further avoid overfitting, we set $m = 0$ and $B = 32$ when all the replay samples in the memory have been used when training for a certain task - that is, for each task, a replay sample is used only once. These hyper-parameters were selected arbitrarily. Note that since we are dealing with an online continual learning setting, the number of epochs is always set to 1.

The class-incremental scenario was defined as starting with two classes and gradually increasing the number of classes one by one, whereas the style-incremental scenario commences with one style. In the class-incremental scenario, we utilized prediction blocks of 16 filters, and the maximum number of classes per prediction block is equal to 3. That is, a new prediction block is instantiated at every three new classes. In the style-incremental scenario, one sole prediction block of 64 filters was employed.

4.5 Comparison with Baselines

For comparison, we implemented EWC [40], LWF [27], GEM [28], AGEM [5], and PNN [37] on the microcontrollers. Since PNNs [37] are only designed for an increasing number of outputs, it is not used in the style-incremental scenario, whose number of outputs (predicted classes) remains constant. Moreover, we also present results of **fine-tuning** as a continual learning method and **joint training**. Fine-tuning is the most simplistic approach to continual learning and it consists in training the neural network with a monotonically decreasing learning rate and/or by freezing

Table 3. Results the Class-Incremental Scenario Where Our Method is Compared with the Baselines for all Datasets based on the Average Accuracy and Backward Transfer

	Average accuracy (ACC)				Backward transfer (BWT)			
	PAMAP2	OPP.	Skoda	MHEALTH	PAMAP2	OPP.	Skoda	MHEALTH
Joint training	0.860	0.761	0.919	0.840	–			
Fine-tuning	0.240	0.128	0.266	0.236	0.098	−0.020	0.096	−0.054
EWC [40]	0.280	0.200	0.349	0.523	0.036	0.085	0.125	0.098
LwF [27]	0.248	0.204	0.255	0.212	0.109	0.070	0.088	−0.021
GEM [28]	0.468	0.302	0.880	0.655	0.096	0.037	0.188	0.037
AGEM [5]	0.443	0.217	0.453	0.566	0.166	0.071	0.208	0.199
PNN [37]	0.369	0.131	0.323	0.425	0.092	0.045	0.108	0.099
Ours	0.746	0.530	0.700	0.689	0.301	0.192	0.282	0.155

OPP stands for Opportunity.

early layers, as well as adding new neurons to the last layer in case new classes are required to be recognized (class-incremental scenario). In our implementation of fine-tuning, the learning rate is reduced gradually as new tasks are learned. Joint training refers to the case where the classifier is trained simultaneously with the data of all tasks for multiple epochs - i.e. this is the standard learning case.

In AGEM [5], larger (smaller) sizes of the randomized subset of the replay memory lead to potentially better (worse) performance and consume resources to a greater (smaller) extent. To balance this trade-off, we set the size of the randomized subset to be equal to half of the replay memory. For EWC [40], we approximate the Fisher information matrix with the square of the logarithm of the probabilities given at the output of the neural network.

For all baselines, to guarantee a fair comparison, we utilized a neural network with the same architecture as ours. The number of filters was set to 64 for all the baselines with the exception of PNN [37]. This number of filters - as well as the hyper-parameters in EWC [40] and LwF [27] - was verified to be the best performing based on a trial and error search with the validation set. As for PNN [37], similar to our neural network, a prediction block with 16 filters and lateral connections to the previous blocks is instantiated at every three classes. The batch size for all the baselines was also set to 32 with the same learning rate and optimizer configurations as used for our method.

5 EXPERIMENTS

5.1 Class-Incremental Scenario

Table 3 reports the performance results in terms of the ACC and BWT (Equations (2) and (3), respectively) metrics. Figures 5 and 6 illustrate the evolution of $R_{i,i}$ as i approaches T as well as the resource consumption in terms of memory usage and training duration. It is observed from Table 3 - due to the low values of ACC and BWT - that catastrophic forgetting occurs not only in the fine-tuning approach but also in the LwF [27] and PNN [37] methods. In LwF [27], the knowledge distillation model increasingly accumulates errors as the number of classes grows, suffering from an exponential decrease in test prediction accuracy (see the LwF [27] curves in Figures 5 and 6). With respect to PNN [37], as the number of classes increases, due to the lack of a replay memory, the method struggles at distinguishing between the classes that are not jointly trained - this is known as **inter-task confusion** [31] and is seen by the low values of ACC. EWC [40] offers slightly better performance, but is still very limited due to the trade-off in weights between old and new tasks.

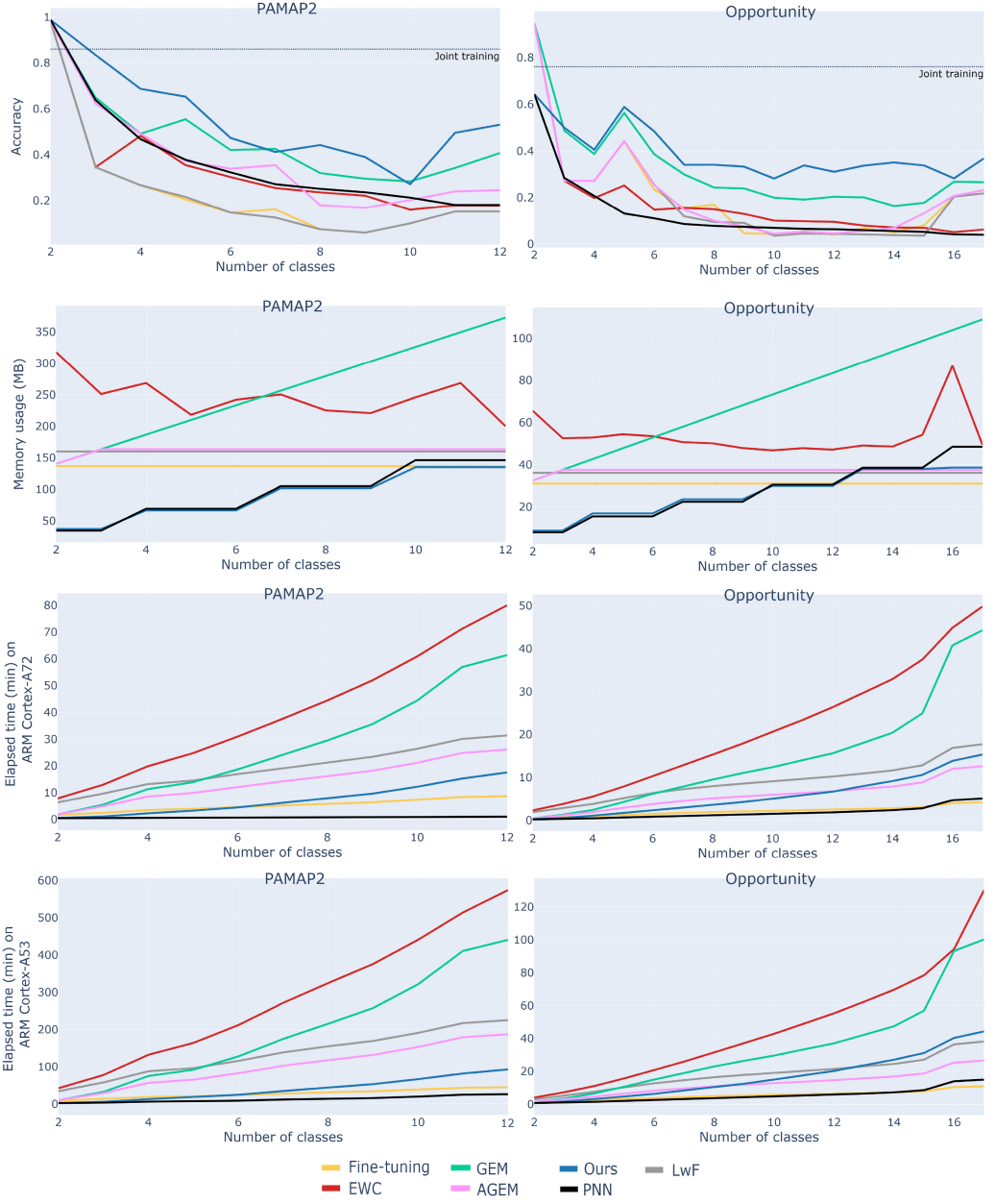


Fig. 5. First row: graphs of the prediction accuracy $R_{i,i}$ as the number of classes grow (i.e. i approaches T). Second row: memory usage on both microcontrollers. Third row: elapsed time of the training on the ARM Cortex-A72. Fourth row: elapsed time of the training on the ARM Cortex-A53. PAMAP2 and Opportunity datasets. Best seen in color.

Better results are seen with AGEM [5] as it is able to restrain the forgetting up to a certain extent (notice the ACC and BWT metrics in Table 3). However, it chooses to populate the replay memory with the last seen samples of each class. In practice, for HAR, this signifies that the replay memory has a low variability of ways (styles) the activities can be performed - that is, the samples

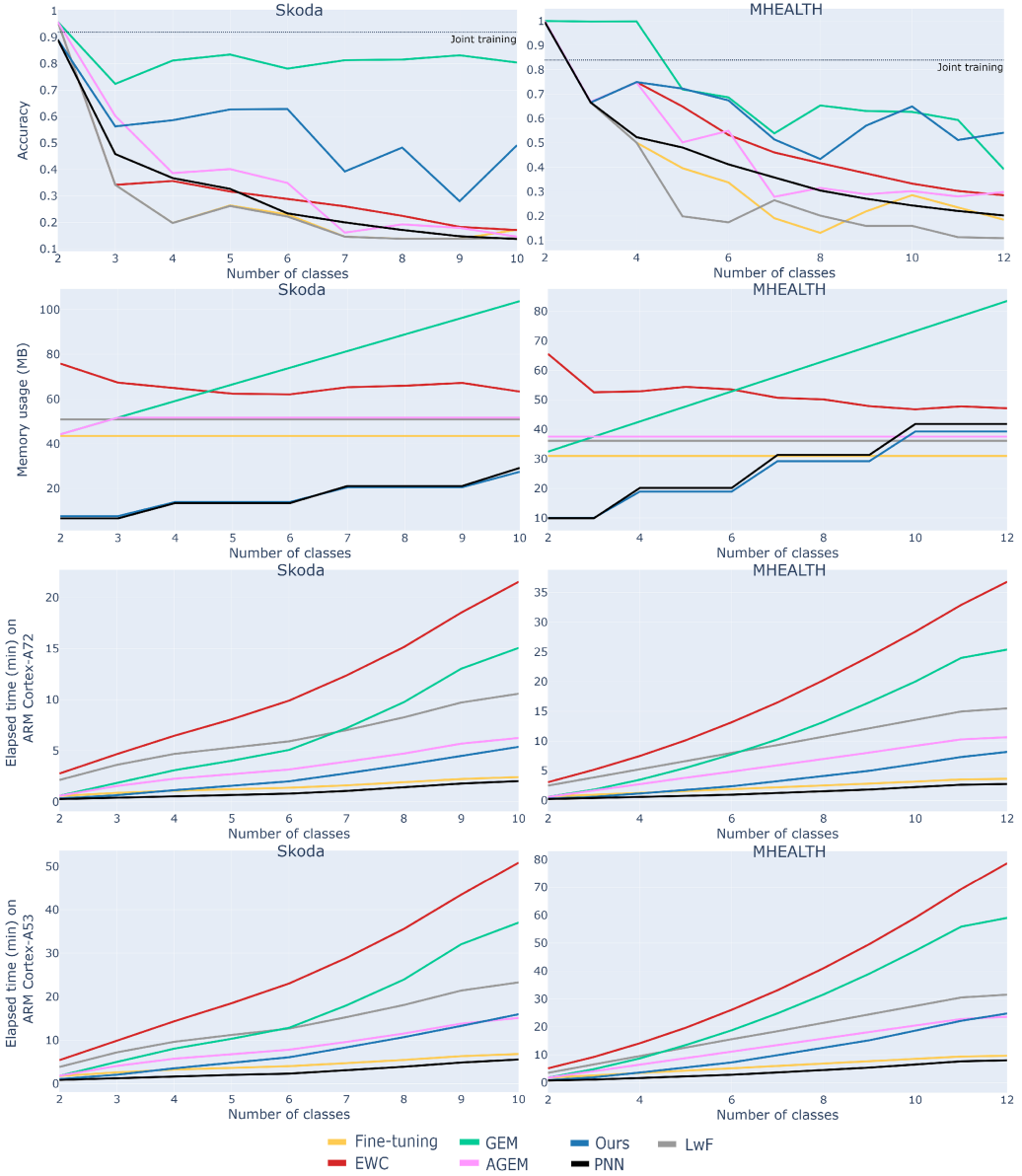


Fig. 6. First row: graphs of the prediction accuracy $R_{i,i}$ as the number of classes grow (i.e. i approaches T). Second row: memory usage on both microcontrollers. Third row: elapsed time of the training on the ARM Cortex-A72. Fourth row: elapsed time of the training on the ARM Cortex-A53. Best seen in color.

of the replay memory contain similar information. This not only leads the neural network to forget different ways the activities can be performed but also to overfit the limited styles present in the replay memory. GEM [28] employs the same selection criterion for its replay memory. However, it uses the entire replay memory at each optimization step, thus, being able to more effectively prevent forgetting.

Our method achieves better results compared with the second best-performing method GEM [28]. As a result of the downsampling and precision reduction, we are able to store and utilize $16\times$ more samples in the replay memory than the other methods. This not only circumvents catastrophic forgetting but also combats the inter-task confusion when the neural network is expanded. The better performance of our method can be also explained by the expected variability of the selected samples. These two factors enable a superior performance compared to complex and resource-demanding methods as GEM [28].

We measured the training duration and memory footprint when running our method and the baselines on the two microcontrollers, including the Raspberry Pi 4B and the Raspberry Pi 3B+. The results are shown in Figures 5 and 6. With respect to the memory footprint, there is no distinction across microcontrollers. Therefore, the following comments regarding resource utilization apply to both microcontrollers. In EWC [40], the importance weights - Fisher information matrix - are calculated at the end of each task utilizing all the data pertaining to the task in question, this incurs high memory cost since a large number of samples is required to be stored in memory until the end of the task. The calculation of the importance weights is also responsible for the enormous duration of the training as it consists in performing numerous backward passes on the neural network. Given that each task has an equal number of samples, the memory footprint for EWC [40] remains constant and the training duration increases linearly. In LwF [27], the samples can be discarded as soon as the neural network is trained with them, which reduces the memory cost considerably. The computational costs remain constant as the number of tasks increases, hence the training duration also grows linearly.

However, this is not the case for GEM [28]. GEM [28] solves a quadratic optimization problem with a number of constraints equal to the number of tasks. The constraints are formed with the gradients calculated with the replay memory of each task. This leads to an ever-increasing computational cost and memory footprint. AGEM [5] abandons the idea of a growing number of constraints in favor of one sole constraint, this guarantees constant memory usage and approximately equal training duration per task.

Our method works with a significantly lower number of backward passes that remains constant as the amount of tasks grows. Also, when the neural network is expanded, the computational costs do not increase substantially. For these reasons, our method exhibits a linear growth in the elapsed training duration. Concerning memory usage, the results of our method are significantly lower than non-expandable neural networks. An expandable neural network does not need to start with a high complexity as a future-proofing method. Instead, the complexity grows as new classes are required to be learned. This results in more efficient memory usage. Eventually, our neural network can reach a memory usage higher than the static neural networks utilized in the other methods (except for PNNs). However, the increase in memory utilization is minimal and means that the neural network does still have the capacity to learn entirely new features important to the recognition of new classes. Static neural networks simply become saturated and commence to underfit the data.

Our method employs operations not present in other continual learning methods. These operations refer to the sample identifier storage, the downsampling of samples before their storage in the replay memory, their precision reduction, and lastly their interpolation before rehearsal (i.e. performing backward passes with replay samples). The former utilizes negligible processing cost and memory space (less than 3e-9% of the total memory usage). The last three mentioned operations account for less than 10% of the total processing expenses of our method. Hence, most of the processing costs are assigned to the backward passes on the neural network. Note that all four operations are only present during the training.

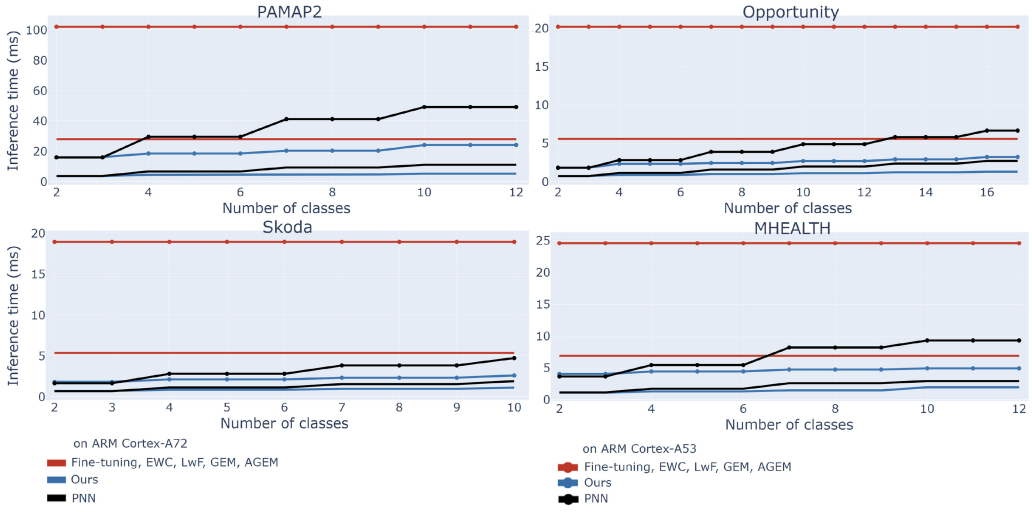


Fig. 7. Comparison of inference time in the class-incremental scenario among our model, PNN [37], and static models utilized in the fine-tuning, EWC [40], LwF [27], GEM [28], and AGEM [5] methods. The lines with circles represent the results on the ARM Cortex-A53 chip, whereas those without circles refer to the ARM Cortex-A72 one. Best seen in color.

PNNs have a similar memory cost compared to our neural network. However, as more lateral connections are instantiated, the computational graph rises relatively more in complexity, thus, requiring a higher memory cost than our method. This increase in complexity is not accompanied by any performance improvement. PNNs are still able to train faster since backward passes do not happen in the frozen portion of the neural network. However, this also contributes to inter-task confusion and, consequently, catastrophic forgetting.

The inference results are illustrated in Figure 7. For all datasets, our model is able to achieve significantly lower inference time compared to the static model utilized in the continual learning baselines (with the exception of PNNs [37]). This is a consequence of the simpler computational graph where features of distinct groups of classes are learned separately. Despite the fact that PNNs [37] have the same number of weights as in our model, its inference time is longer than our model due to the lateral connections between the expanded blocks. Notice that the results for the ARM Cortex-A72 are similar to those of the ARM Cortex-A53, with the exception that the former can execute the inference 3–4× faster than the latter. Additionally, we have evaluated the performance degradation when the model’s parameters are quantized to an 8-bit fixed-point representation before being deployed to the microcontroller. In such a case, the decrease in the F1-score remained below 2%, whereas the model’s size was reduced fourfold. Libraries such as TensorFlow Lite takes advantage of fixed-point quantization to provide 3x+ speedup in the inference of models. Note that these quantities are independent of the continual learning method applied.

5.2 Style-Incremental Scenario

Table 4 exhibits the results in terms of the ACC, BWT. Figure 8 reports the values of $R_{i,i}$, and resource consumption metrics. Since the data distribution across different styles varies on a much smaller scale than across classes, the forgetting is significantly less pronounced in the style-incremental scenario. This results in an accuracy that either slowly grows or oscillates (see the evolution of the curve in Figure 8) in the fine-tuning approach, thus making it hard to spot any

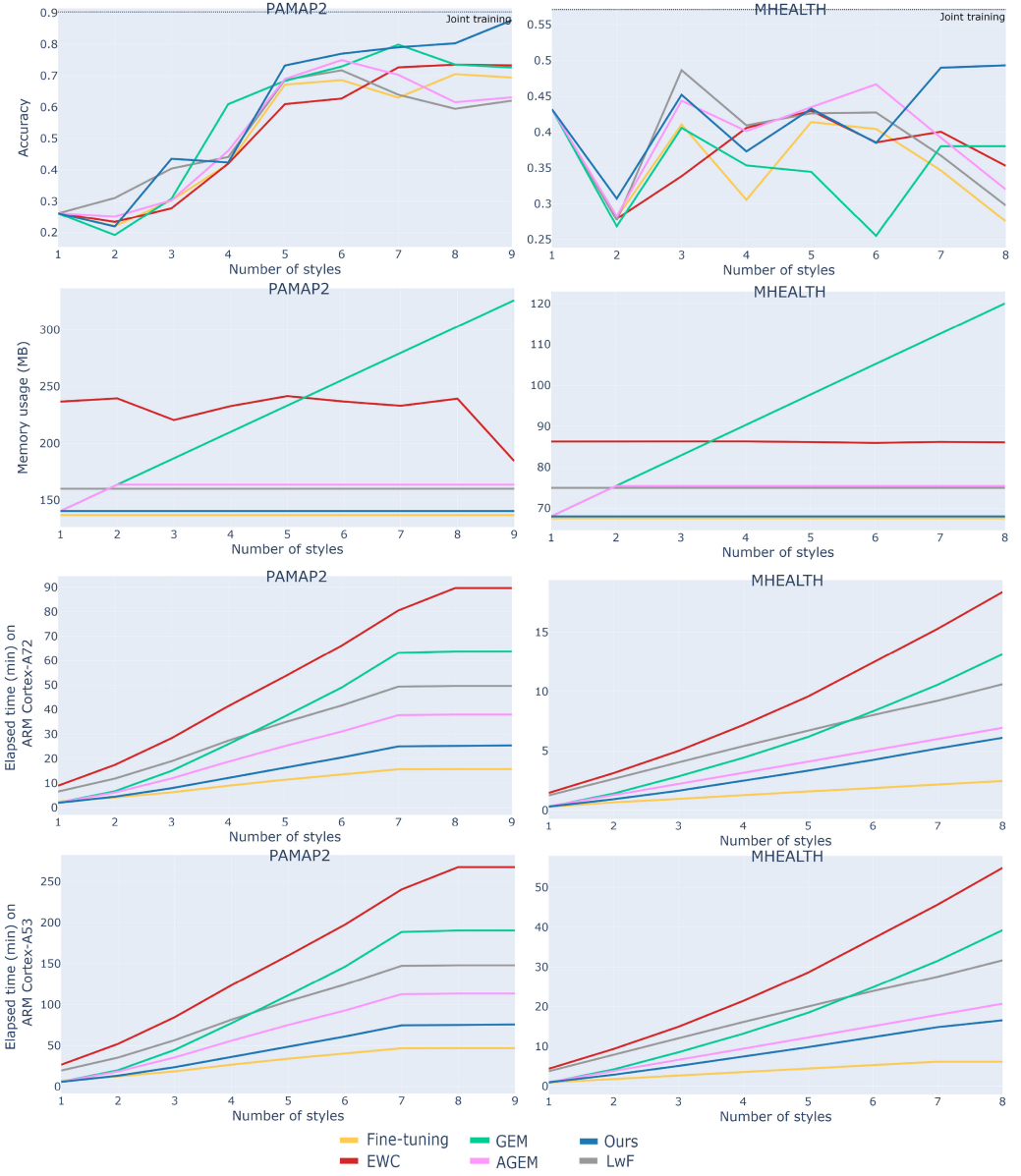


Fig. 8. First row: graphs of the prediction accuracy $R_{i,i}$ as the number of styles grow (i.e. i approaches T). Second row: memory usage on both microcontrollers. Third row: elapsed time of the training on the ARM Cortex-A72. Differences with respect to the training duration are hardly noticeable across the microcontrollers. Fourth row: elapsed time of the training on the ARM Cortex-A53. Best seen in color.

forgetting. Nevertheless, forgetting is still present in the fine-tuning approach since the continual learning methods - in special, our method - result in higher ACC values.

We also observe that the continual learning baselines provide close to negligible - or even detrimental - performance improvement compared to the fine-tuning approach. This indicates that these methods are not appropriate for scenarios where the variation in the data distribution

Table 4. Results of Style-Incremental Scenario Where Our Method is Compared with the Baselines Based on the Average Accuracy and Backward Transfer

	Average accuracy (ACC)		Backward transfer (BWT)	
	PAMAP2	MHEALTH	PAMAP2	MHEALTH
Joint training	0.903	0.571	–	
Fine-tuning	0.723	0.330	0.272	0.030
EWC [40]	0.732	0.387	0.278	0.072
LwF [27]	0.766	0.341	0.253	0.005
GEM [28]	0.738	0.293	0.232	–0.005
AGEM [5]	0.662	0.367	0.194	0.027
Ours	0.898	0.468	0.379	0.116

reduces across tasks. On the contrary, our method is able to deliver an appreciable leap in performance with respect to fine-tuning since our design is heavily grounded on a rich - yet small in memory footprint - replay memory. Concerning the resource consumption, the observations made on the EWC [40], LwF [27], GEM [28], and AGEM [5] methods are still valid in the style-incremental scenario. Since our model shares the same neural network architecture as the baselines, the inference time is exactly the same for all methods in the style-incremental scenario: 27.78ms and 6.91ms for PAMAP2 and MHEALTH, respectively, running on the ARM Cortex-A72. Again, the results on the Cortex A-53 are approximately 3-4× slower.

Although we have utilized four datasets with distinct characteristics - such as number of participants, number of classes, number of sensors, and sampling rate - the results exhibited consistent across them for both scenarios. This is not only true for our method, but also for the baselines.

5.3 Ablation Studies

To evaluate our design choices, we perform ablation experiments with two variants: Ours-L and Ours-URP. The former variant Ours-L replaces our sample selection algorithm with that proposed in [5, 28], i.e. only the last seen samples populate the replay memory. The latter variant utilizes a replay memory with unmodified samples - i.e. with the original sampling rate and precision.

Tables 5 and 6 delineate the results of the ablation studies for the class-incremental and style-incremental scenarios, respectively. Across all scenarios and datasets, it is observed that the down-sampled and precision-reduced replay memory contributes the most to the high performance of our method, providing on average a leap of 24.7% in the accuracy compared to a replay memory of equal size with unmodified samples. Our sample selection algorithm also plays an important role in the accuracy results since it contributes to an accuracy improvement greater than 10%. These results evidence the importance of our design choices.

6 DISCUSSION

In this section, we discuss the advantages, drawbacks, and future work related to our method.

6.1 Advantages

Our proposal is the first to address continual learning for sensor-based human activity recognition on resource-constrained devices and it provides the following advantages.

High performance. Our method is capable of delivering similar and - in some cases - even better performance than GEM [28] and better performance than the other methods such as EWC [40], LwF [27], PNN [37], and AGEM [5]. This is due to an efficient replay memory with a large number

Table 5. Results of the Ablation Studies for the Class-Incremental Scenario

	Average accuracy (ACC)				Backward transfer (BWT)			
	PAMAP2	Opportunity	Skoda	MHEALTH	PAMAP2	Opportunity	Skoda	MHEALTH
Ours	0.746	0.530	0.700	0.689	0.301	0.192	0.282	0.155
Ours-L	0.667	0.396	0.678	0.456	0.224	0.095	0.263	0.222
Ours-URP	0.426	0.215	0.421	0.420	0.153	0.037	0.151	0.119

Table 6. Results of the Ablation Studies for the Style-Incremental Scenario

	Average accuracy (ACC)		Backward transfer (BWT)	
	PAMAP2	MHEALTH	PAMAP2	MHEALTH
Ours	0.898	0.468	0.379	0.116
Ours-L	0.774	0.368	0.173	0.043
Ours-URP	0.708	0.358	0.174	0.026

of samples - although with equal memory footprint as other methods - with rich diversity among them.

Low resource consumption. Our method has a low memory usage and processing power in both training and inference phases, thus, being able to better fit in low-cost resource-constrained devices. This can be explained by the following points.

- Since our neural network architecture is expandable as the number of tasks (in the class-incremental scenario, each task represents an additional class to be learned) grows, it is possible to start with a fairly small and lightweight architecture. In methods that use a static neural network architecture, one must start with a considerably large architecture to account for future tasks. Eventually, our neural network architecture may reach a level of complexity higher (and, therefore, utilize greater computational resources during inference) than the static model. However, when this happens, it also means that the static model has reached its maximum capacity. To proceed with accommodating new knowledge, the static model is required to grow in complexity.
- Our method requires significantly fewer backward passes during the training, compared with AGEM [5] and GEM [28] since these methods make use of additional computations of the gradient to curb forgetting. Also, unlike GEM, our method does not require the solution of a multitude of constrained quadratic optimization problems during the training. These problems not only increase processing costs but also memory costs.
- Particularly for EWC [40], processing costs can be significantly high since, after a task τ has been trained, to obtain the importance of the weights of τ , the diagonal of Fisher information matrix is calculated by performing one backward pass with the entire training set of τ . This also results in a large memory cost since the training samples of the task τ cannot be discarded before the calculation of the importance weights. Our method avoids these two issues, i.e. large memory and processing costs. Our method does not require the storage of the entire training set of a certain task, thus avoiding elevated memory usage.
- The loss function utilized in LwF [27] and EWC [40] presents an additional term - a knowledge distillation term in LwF [27] and a weight regularization term in EWC [40] - that adds to the training duration. Also, with respect to LwF [27], in addition to the backward passes performed to optimize the weights of the training neural network, forward passes on the distillation model are also required, which increases the computational costs during

training. The storage of importance weights (in EWC) and the distillation model (in LwF [27]) increases the memory costs of these methods.

- Our method utilizes a lightweight replay memory since the storage samples undergo a process of downsampling and precision reduction. Since the embedded device has a rather limited memory capacity, a lightweight replay memory is beneficial. This is even more critical in deep learning settings, where the input data usually contain dozens - or even hundreds - of sensors sampling at a high frequency of tens of times per second.

Consistency in hyper-parameter choice. Methods such as EWC [40] and LwF [27] require the careful selection of a constant that balances the terms in their loss function. Different datasets were shown to work better with different values of this hyper-parameter. Our method, on the opposite, showed consistency across datasets with respect to its hyper-parameters, thus, excluding possible overheads in hyper-parameter selection. We posit that the same hyper-parameters used in our method for all datasets studied can be utilized for any other dataset. Note that since we are dealing with an online continual learning scenario, a careful selection or optimization of hyper-parameters is not possible. The reason is two-fold. First, each training sample is only used once - except if the sample is selected to populate the replay memory - and is discarded due to memory constraints. Hyper-parameter selection methods - such as grid search or Taguchi's experimental method [11] - requires the utilization of the training samples multiple times. Second, due to limited computational resources, running multiple instances of training with different hyper-parameters is prohibitively expensive.

Absence of neural network compression. The weights of the neural network in our method do not undergo any type of lossy compressions such as pruning, quantization and factorization. Hence, the neural network after the last training step and during the inference is exactly the same. More specifically, the data types utilized in the parameters (weights and biases) of the neural network are always floating-point numbers.

6.2 Drawbacks and Future Work

Extreme task imbalance. When the stream of data for a certain task τ is extremely larger than for previous tasks, catastrophic forgetting may occur. Re-utilizing the same replay samples throughout the duration of τ as a way to compensate may cause the neural network to overfit those samples. This overfitting effect can be circumvented utilizing GEM [28] or AGEM [5]; however, these methods are computationally expensive. Thus, a future direction of research consists of dealing with task imbalance at a low cost.

Replay memory. The replay memory is the essential part of our method. Its significance is that of curbing the catastrophic forgetting induced by accommodating new knowledge into the neural network when learning new tasks. In some cases, replay samples may be not available, which renders impossible the utilization of our method. Therefore, it is important to devise ways to either recover replay data directly from the weights of the neural network or to combine our method with parameter isolation or regularization approaches to cover the cases of inaccessible replay samples.

Our replay memory applied to other methods. It is possible that replay-based methods - such as AGEM [5], GEM [28], and GSS [3] - can be modified to utilize a replay memory with downsampled and precision-reduced samples as in our method. However, doing so would also mean that the replay memory could be populated with a larger number of samples and this would make the computational costs for these replay-based methods skyrocket. For example, in the particular case of GEM [28], a larger replay memory would also increase the number of variables in the quadratic optimization problem that is executed at every training step.

7 CONCLUSIONS

We have proposed a method that, to the best of our knowledge, is the first to offer a resource-efficient solution for continual learning scenarios in HAR. It was founded on two basic premises: (1) highly compressed rehearsal data can successfully circumvent catastrophic forgetting, and (2) maximizing the spread of the rehearsal (replay) data along time provides them with a rich diversity which is essential for high-performance continual learning HAR. Our method was applied to two continual learning scenarios in HAR: class-incremental and style-incremental. Compared to continual learning baselines, our proposal was shown to deliver impressive performance leaps over state-of-the-art methods such as GEM [28] while utilizing one-third of the memory and being up to $3\times$ faster.

REFERENCES

- [1] M. Abdel-Basset, H. Hawash, R. K. Chakraborty, M. Ryan, M. Elhoseny, and H. Song. 2021. ST-DeepHAR: Deep learning model for human activity recognition in IoHT applications. *IEEE Internet of Things Journal* 8, 6 (2021), 4969–4979. <https://doi.org/10.1109/JIOT.2020.3033430>
- [2] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars. 2017. Memory aware synapses: Learning what (not) to forget. *CoRR abs/1711.09601* (2017). arXiv:1711.09601 <http://arxiv.org/abs/1711.09601>.
- [3] R. Aljundi, M. Lin, B. Goujaud, and Y. Bengio. 2019. Gradient based sample selection for online continual learning. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2019/file/e562cd9c0768d5464b64cf61da7fc6bb-Paper.pdf>.
- [4] O. Banos, J. A. Moral-Munoz, I. Diaz-Reyes, M. Arroyo-Morales, M. Damas, E. Herrera-Viedma, C. S. Hong, S. Lee, H. Pomares, I. Rojas, and C. Villalonga. 2015. mDurance: A novel mobile health system to support trunk endurance assessment. *Sensors* 15, 6 (2015), 13159–13183.
- [5] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny. 2019. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations*.
- [6] R. Chavarriaga, H. Sagha, A. Calatroni, S. T. Digumarti, G. Tröster, J. del R. Millán, and D. Roggen. 2013. The opportunity challenge: A benchmark database for on-body sensor-based activity recognition. *Pattern Recognition Letters* 34, 15 (2013), 2033–2042.
- [7] K. Chen, D. Zhang, L. Yao, B. Guo, Z. Yu, and Y. Liu. 2021. Deep learning for sensor-based human activity recognition: Overview, challenges, and opportunities. *ACM Comput. Surv.* 54, 4, Article 77 (May 2021), 40 pages. <https://doi.org/10.1145/3447744>
- [8] M. Delange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars. 2021. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2021), 1–1.
- [9] S. Dhar, J. Guo, J. Liu, S. Tripathi, U. Kurup, and M. Shah. 2021. A survey of on-device machine learning: An algorithms and learning theory perspective. *ACM Trans. Internet Things* 2, 3, Article 15 (July 2021), 49 pages.
- [10] Y. Du, Y. Lim, and Y. Tan. 2019. A novel human activity recognition and prediction in smart home based on interaction. *Sensors* 19, 20 (2019), 1–16.
- [11] S. Gao, M. Zhou, Y. Wang, J. Cheng, H. Yachi, and J. Wang. 2019. Dendritic neuron model with effective learning algorithms for classification, approximation, and prediction. *IEEE Transactions on Neural Networks and Learning Systems* 30, 2 (2019), 601–614. <https://doi.org/10.1109/TNNLS.2018.2846646>
- [12] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. 2014. Generative Adversarial Networks. arXiv:1406.2661 [stat.ML]
- [13] F. Gu, M. Chung, M. Chignell, S. Valae, B. Zhou, and X. Liu. 2021. A survey on deep learning for human activity recognition. *ACM Comput. Surv.* 54, 8, Article 177 (Oct. 2021), 34 pages. <https://doi.org/10.1145/3472290>
- [14] Y. Guan and T. Plötz. 2017. Ensembles of deep LSTM learners for activity recognition using wearables. *IMWUT* 1, 2, Article 11 (June 2017), 28 pages.
- [15] J. Haladjian, D. Schlabbbers, S. Taheri, M. Tharr, and B. Bruegge. 2020. Sensor-based detection and classification of soccer goalkeeper training exercises. *ACM Trans. Internet Things* 1, 2, Article 12 (April 2020), 20 pages.
- [16] N. Y. Hammerla, S. Halloran, and T. Plötz. 2016. Deep, convolutional, and recurrent models for human activity recognition using wearables. In *IJCAI'16*. AAAI Press, 1533–1540.
- [17] M. M. Hassan, A. Gumaee, G. Aloï, G. Fortino, and M. Zhou. 2019. A smartphone-enabled fall detection framework for elderly people in connected home healthcare. *IEEE Network* 33, 6 (2019), 58–63. <https://doi.org/10.1109/MNET.001.1900100>

- [18] G. Hinton, O. Vinyals, and J. Dean. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531 [stat.ML]
- [19] J. Hsiao, Y. Deng, T. Pao, H. Chou, and J. Chang. 2017. Design of a wireless 3D hand motion tracking and gesture recognition glove for virtual reality applications, Vol. ASME 2017 Conference on Information Storage and Processing Systems. 1–3.
- [20] S. Jha, M. Schiemer, F. Zambonelli, and J. Ye. 2021. Continual learning in sensor-based human activity recognition: An empirical benchmark analysis. *Information Sciences* (April 2021).
- [21] L. Jin, X. Wang, J. Chu, and M. He. 2022. Human activity recognition machine with an anchor-based loss function. *IEEE Sensors Journal* 22, 1 (2022), 741–756. <https://doi.org/10.1109/JSEN.2021.3130761>
- [22] J. Kirkpatrick, R. Pascanu, N. C. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. 2016. Overcoming catastrophic forgetting in neural networks. *CoRR* abs/1612.00796 (2016). arXiv:1612.00796 <http://arxiv.org/abs/1612.00796>.
- [23] N. D. Lane, S. Bhattacharya, P. Georgiev, C. Forlivesi, L. Jiao, L. Qendro, and F. Kawsar. 2016. DeepX: A software accelerator for low-power deep learning inference on mobile devices. In *2016 15th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*. 1–12. <https://doi.org/10.1109/IPSN.2016.7460664>
- [24] C. F. S. Leite and Y. Xiao. 2020. Improving cross-subject activity recognition via adversarial learning. *IEEE Access* 8 (2020), 90542–90554.
- [25] C. F. S. Leite and Y. Xiao. 2020. Improving resource efficiency of deep activity recognition via redundancy reduction. In *HotMobile'20*. Association for Computing Machinery, 33–38.
- [26] C. F. S. Leite and Y. Xiao. 2021. Optimal sensor channel selection for resource-efficient deep activity recognition. In *Proceedings of the 20th International Conference on Information Processing in Sensor Networks (Co-Located with CPS-IoT Week 2021)* (Nashville, TN, USA) (IPSN'21). Association for Computing Machinery, New York, NY, USA, 371–383.
- [27] Z. Li and D. Hoiem. 2016. Learning without forgetting. *CoRR* abs/1606.09282 (2016). arXiv:1606.09282 <http://arxiv.org/abs/1606.09282>.
- [28] D. Lopez-Paz and M. Ranzato. 2017. Gradient episodic memory for continuum learning. *CoRR* abs/1706.08840 (2017). arXiv:1706.08840 <http://arxiv.org/abs/1706.08840>.
- [29] D. Ludl, T. Gulde, and C. Curio. 2020. Enhancing data-driven algorithms for human pose estimation and action recognition through simulation. *IEEE Transactions on Intelligent Transportation Systems* 21, 9 (2020), 3990–3999. <https://doi.org/10.1109/TITS.2020.2988504>
- [30] A. Mallya and S. Lazebnik. 2017. PackNet: Adding multiple tasks to a single network by iterative pruning. *CoRR* abs/1711.05769 (2017). arXiv:1711.05769 <http://arxiv.org/abs/1711.05769>.
- [31] M. Masana, X. Liu, B. Twardowski, M. Menta, A. D. Bagdanov, and J. van de Weijer. 2020. Class-incremental learning: Survey and performance evaluation. *CoRR* abs/2010.15277 (2020). arXiv:2010.15277 <https://arxiv.org/abs/2010.15277>.
- [32] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida. 2018. Spectral normalization for generative adversarial networks. *CoRR* abs/1802.05957 (2018). arXiv:1802.05957 <http://arxiv.org/abs/1802.05957>.
- [33] F. J. Ordóñez and D. Roggen. 2016. Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition. *Sensors* 16, 1 (2016), 1–25.
- [34] N. Rashid, B. U. Demirel, and M. A. Al Faruque. 2021. AHAR: Adaptive CNN for energy-efficient human activity recognition in low-power edge devices. *CoRR* abs/2102.01875 (2021). arXiv:2102.01875 <https://arxiv.org/abs/2102.01875>.
- [35] S. Rebuffi, A. Kolesnikov, and C. H. Lampert. 2016. iCARL: Incremental classifier and representation learning. *CoRR* abs/1611.07725 (2016). arXiv:1611.07725 <http://arxiv.org/abs/1611.07725>.
- [36] A. Reiss and D. Stricker. 2012. Introducing a new benchmarked dataset for activity monitoring. In *2012 16th International Symposium on Wearable Computers*. 108–109.
- [37] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. 2016. Progressive neural networks. *CoRR* abs/1606.04671 (2016). arXiv:1606.04671 <http://arxiv.org/abs/1606.04671>.
- [38] L. Schrader, A. Vargas Toro, S. Konietzny, S. Rüping, B. Schäpers, M. Steinböck, C. Krewer, F. Müller, J. Güttler, and T. Bock. 2020. Advanced sensing and human activity recognition in early intervention and rehabilitation of elderly people. *Journal of Population Ageing* 13, 2 (June 2020), 139–165.
- [39] H. Shin, J. Kwon Lee, J. Kim, and J. Kim. 2017. Continual learning with deep generative replay. *CoRR* abs/1705.08690 (2017). arXiv:1705.08690 <http://arxiv.org/abs/1705.08690>.
- [40] J. Thorne and A. Vlachos. 2020. Avoiding catastrophic forgetting in mitigating model biases in sentence-pair classification with elastic weight consolidation. *CoRR* abs/2004.14366 (2020). arXiv:2004.14366 <https://arxiv.org/abs/2004.14366>.
- [41] J. Wang, Y. Chen, S. Hao, X. Peng, and L. Hu. 2019. Deep learning for sensor-based activity recognition: A survey. *Pattern Recognition Letters* 119 (2019), 3–11. <https://doi.org/10.1016/j.patrec.2018.02.010>
- [42] B. Wu, J. Zhong, and C. Yang. 2022. A visual-based gesture prediction framework applied in social robots. *IEEE/CAA Journal of Automatica Sinica* 9, 3 (2022), 510–519. <https://doi.org/10.1109/JAS.2021.1004243>
- [43] C. Xu, D. Chai, J. He, X. Zhang, and S. Duan. 2019. InnoHAR: A deep neural network for complex human activity recognition. *IEEE Access* 7 (2019), 9893–9902.

- [44] Z. Yang, O. Ian Raymond, C. Zhang, Y. Wan, and J. Long. 2018. DFTerNet: Towards 2-bit dynamic fusion networks for accurate human activity recognition. *CoRR* abs/1808.04228 (2018). arXiv:1808.04228 <http://arxiv.org/abs/1808.04228>.
- [45] J. Ye and E. Callus. 2020. Evolving models for incrementally learning emerging activities. *Journal of Ambient Intelligence and Smart Environments* 12, 4 (2020), 313–325..
- [46] J. Ye, P. Nakwijit, M. Schiemer, S. Jha, and F. Zambonelli. 2021. Continual activity recognition with generative adversarial networks. *ACM Trans. Internet Things* 2, 2, Article 9 (March 2021), 25 pages.
- [47] P. Zappi, T. Stiefmeier, E. Farella, D. Roggen, L. Benini, and G. Troster. 2007. Activity recognition from on-body sensors by classifier fusion: Sensor scalability and robustness. In *2007 3rd International Conference on Intelligent Sensors, Sensor Networks and Information*. 281–286.
- [48] M. Zeng, H. Gao, T. Yu, O. J. Mengshoel, H. Langseth, I. Lane, and X. Liu. 2018. Understanding and improving recurrent networks for human activity recognition by continuous attention. In *ISWC'18* (Singapore). ACM, 56–63.
- [49] F. Zenke, B. Poole, and S. Ganguli. 2017. Improved multitask learning through synaptic intelligence. *CoRR* abs/1703.04200 (2017). arXiv:1703.04200 <http://arxiv.org/abs/1703.04200>.
- [50] Y. Zhang, T. Gu, and X. Zhang. 2020. MDLdroidLite: A release-and-inhibit control approach to resource-efficient deep neural networks on mobile devices. *IEEE Transactions on Mobile Computing*. 1 (2020), 1–1. <https://doi.org/10.1109/TMC.2021.3062575>
- [51] Y. Zhang, Z. Zhang, Y. Zhang, J. Bao, Y. Zhang, and H. Deng. 2019. Human activity recognition based on motion sensor using U-net. *IEEE Access* 7 (2019), 75213–75226. <https://doi.org/10.1109/ACCESS.2019.2920969>
- [52] G. Zheng. 2021. A novel attention-based convolution neural network for human activity recognition. *IEEE Sensors Journal* 21, 23 (2021), 27015–27025. <https://doi.org/10.1109/JSEN.2021.3122258>

Received 15 July 2021; revised 6 April 2022; accepted 7 April 2022